```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_bloc_example/models/todo_model.dart';

class TodoCubit extends Cubit<List<Todo>> {
  TodoCubit() : super([]);

  void addTodo(String name) {
    final todo = Todo(name: name, createdAt: DateTime.now());
    // [emit] does nothing if the [state] being emitted is equal to the current
[state].
    emit([...state, todo]);
  }
}
```

==========

```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_bloc_example/bloc/counter_event.dart';

class CounterBloc extends Bloc<CounterEvent, int> {
  CounterBloc() : super(0) {
    on<CounterIncrement>((event, emit) => increment(event, emit));
    on<CounterDecrement>((event, emit) => decrement(event, emit));

    // on<CounterIncrement>(increment);
  }

  void increment(CounterIncrement event, Emitter<int> emit) {
    // Here we can't use emit of the main class like cubit,
    // because we have multiple events to listen to them
    emit(state + 1);
  }

  void decrement(CounterDecrement event, Emitter<int> emit) {
    emit(state - 1);
  }
}
```

==========

```dart
abstract class CounterEvent {}

final class CounterIncrement extends CounterEvent {}

final class CounterDecrement extends CounterEvent {}
```

==========

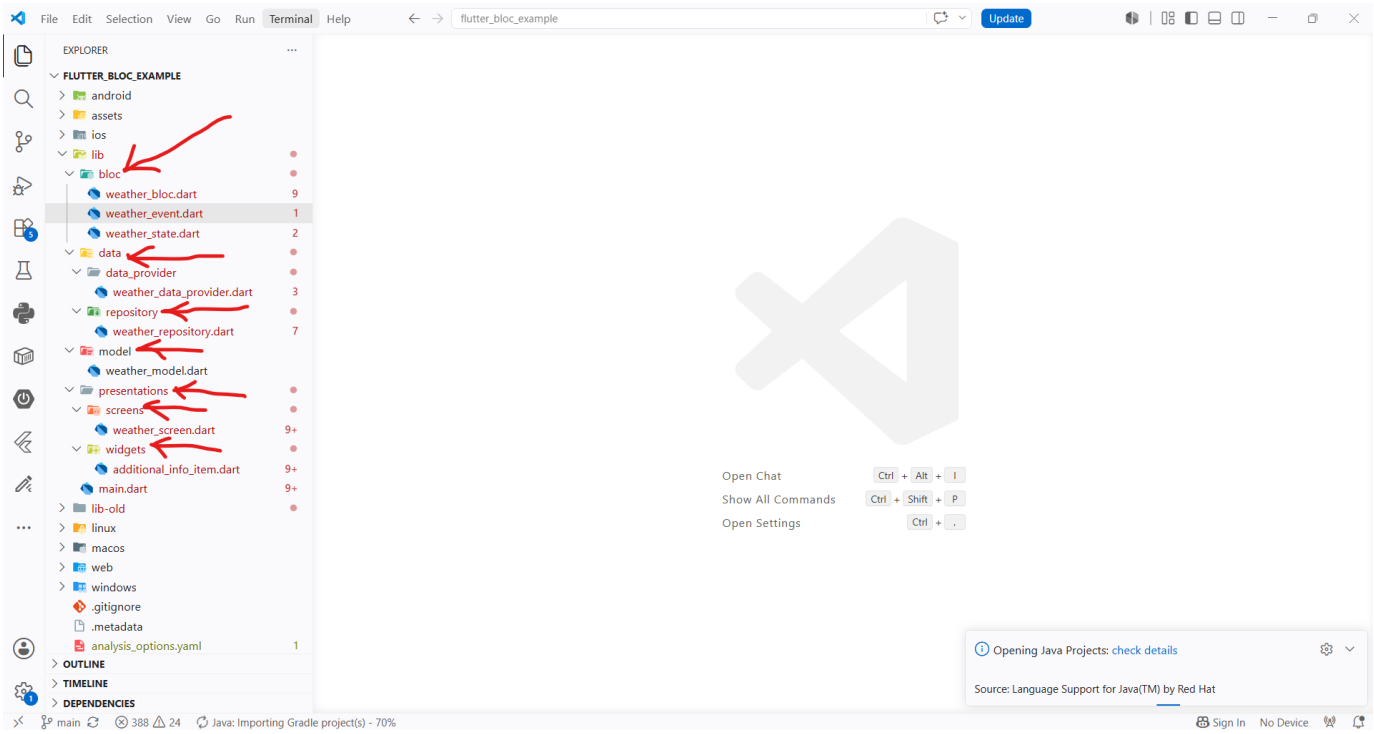
```dart
return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
        BlocProvider(create: (_) => CounterBloc()),
      ],
      child: MaterialApp(
        title: 'Flutter Demo',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(colorScheme: .fromSeed(seedColor: Colors.deepPurple)),
        home: const MyHomePage(title: 'Flutter Demo Home Page'),
      ),
    );
```

```dart
BlocBuilder<CounterBloc, int>(
              // bloc: context.read<CounterCubit>(),
              // bloc: context.read<CounterBloc>(),
              builder: (context, counter) {
                return Text(
                  '$counter',
                  style: Theme.of(context).textTheme.headlineMedium,
                );
              },
),
```

```dart
BlocBuilder<TodoCubit, List<Todo>>(
        builder: (context, state) {
          return ListView.builder(
            itemCount: state.length,
            itemBuilder: (BuildContext context, int index) {
              final item = state[index];
              return ListTile(
                title: Text(item.name),
                trailing: Text(item.createdAt.toIso8601String()),
              );
            },
          );
        },
      ),
```

To define the structure of the project using the Feature Design:

1. We define the bloc where state management logic resides.
2. We define **the data** that contains the:
    a. The **data provider** that calls the APIs endpoints.
    b. The **repository** that uses the data provider.
3. **The model** that contains the data representation for **the request** and **response**.
4. **The presentations** that contain the **screens** and **widgets of our application**.



==========

To define the logic using the bloc state management we define:

1. **The events** of the logic.
2. **The states** of the logic.
3. **The bloc** of the logic that combine the **events** with the **states**.

==========

To build the events we can use classes:

```
@immutable
sealed class WeatherEvent {}

final class WeatherFetched extends WeatherEvent {}
```

To build the states that represents the flow of the logic:

```
@immutable
sealed class WeatherState {}

final class WeatherInitial extends WeatherState {}

final class WeatherSuccess extends WeatherState {
  final WeatherModel weatherModel;
  WeatherSuccess({required this.weatherModel});
}

final class WeatherFailure extends WeatherState {
  final String error;
  WeatherFailure({required this.error});
}

final class WeatherLoading extends WeatherState {}
```

To use the states and events with the bloc:

```
class WeatherBloc extends Bloc<WeatherEvent, WeatherState> {
  final WeatherRepository weatherRepository;
  WeatherBloc({required this.weatherRepository}) : super(WeatherInitial()) {
    on<WeatherFetched>(onWeatherFetched);
  }

  void onWeatherFetched(
    WeatherFetched event,
    Emitter<WeatherState> emit,
  ) async {
    emit(WeatherLoading());
    try {
      final weather = await this.weatherRepository.getCurrentWeather(
        "Damascus",
      );
      emit(WeatherSuccess(weatherModel: weather));
    } catch (e) {
      emit(WeatherFailure(error: e.toString()));
    }
  }
}
```

==========

The simple data provider:

```
class WeatherDataProvider {
  Future<String> getCurrentWeather(String cityName) async {
    try {
      final res = await http.get(
        Uri.parse(
          'https://api.openweathermap.org/data/2.5/forecast?q=$cityName&APPID=$op
enWeatherAPIKey',
        ),
      );

      return res.body;
    } catch (e) {
      throw e.toString();
    }
  }
}
```

==========

The simple data repository

```
class WeatherRepository {
  final WeatherDataProvider weatherDataProvider;

  WeatherRepository({required this.weatherDataProvider});

  Future<WeatherModel> getCurrentWeather(String cityName) async {
    try {
      final body = await this.weatherDataProvider.getCurrentWeather(cityName);
      final data = jsonDecode(body);

      if (data['cod'] != '200') {
        throw 'An unexpected error occurred';
      }
      debugPrint("The data is: ${data['list'][0]}");
      return WeatherModel.fromJson(data['list'][0]);
    } catch (e) {
      throw e.toString();
    }
  }
}
```

==========

To use the repository with the bloc, we can use The RepositoryProvider & BlocProvider:

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // Note: We have also MultiRepositoryProvider
    return RepositoryProvider(
      create: (context) =>
          WeatherRepository(weatherDataProvider: WeatherDataProvider()),
      child: BlocProvider(
        create: (context) =>
            WeatherBloc(weatherRepository: context.read<WeatherRepository>()),
        child: MaterialApp(
          debugShowCheckedModeBanner: false,
          theme: ThemeData.dark(useMaterial3: true),
          home: const WeatherScreen(),
        ),
      ),
    );
  }
}
```

==========